

Graphs & Greedy Algorithms

A Fresher's Manual — Simple Logic, Working Code, Ready Templates

Covers: Activity Selection · Minimum Coins · Kruskal's MST · Prim's MST · Dijkstra
(two ways)

Written for: TalenciaGlobal Mentor-Led Lab questions

Part 1 — Before You Start: Graph Basics in C++

A graph is just a bunch of points (called vertices or nodes) connected by lines (called edges). Think of cities (nodes) connected by roads (edges). If a road has a length or a cost, the edge has a weight.

In C++, the easiest way to store a graph is the adjacency list. For every node, you keep a list of its neighbours. If the edge has a weight, you store the neighbour and the weight together as a pair.

How an adjacency list looks in code

```
vector<vector<pair<int,int>>> adj(n);  
// adj[u] = list of {neighbour, weight} pairs for node u  
  
// adding an undirected edge between u and v with weight w:  
adj[u].push_back({v, w});  
adj[v].push_back({u, w});    // do this line too, because the edge works both ways
```

If the graph has no weights (just connections), drop the weight and use `vector<vector<int>> adj(n);` instead.

`pair<int,int>` — the small box that holds two numbers

A pair just glues two values together. `pair<int,int> x = {5, 10};` means `x.first = 5` and `x.second = 10`. We use it to store `{neighbour, weight}` or `{start, end}` together, so we can sort both numbers at once.

A safe starting template for every problem in this manual

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. Read input
    int n, m;
    cin >> n >> m;

    // 2. Build the graph (only if the problem needs edges)
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    // 3. Do the actual algorithm here

    // 4. Print the answer
    return 0;
}
```

Keep this skeleton in mind. Every question below is just step 3 changing — the read-input and build-graph parts barely change.

Part 2 — Four Templates You Can Reuse for Almost Anything

Memorise the shape of these four blocks. Once you recognise them, every question in this manual (and most graph questions you'll ever see) becomes “fill in the blank”.

Template A — Greedy Pattern (used in Q1, Q2)

Greedy means: sort the items by some rule, then walk through them once, picking an item whenever it is “safe” to pick.

```
sort(items.begin(), items.end()); // sort by the rule that matters

int answer = 0;
for (auto &item : items) {
    if (/* item is safe to take */) {
        // take it
        answer++;
    }
}
```

```
}
}
```

Template B — Union-Find / Disjoint Set (used in Q3)

Union-Find answers one question fast: “are these two nodes already in the same group?” It is the backbone of Kruskal's algorithm.

```
int parent[1000];

// every node starts as its own group leader
for (int i = 0; i < n; i++) parent[i] = i;

// find the leader of x's group (with path compression)
int findSet(int x) {
    if (parent[x] == x) return x;
    return parent[x] = findSet(parent[x]);
}

// merge x's group and y's group into one
void unionSet(int x, int y) {
    parent[findSet(x)] = findSet(y);
}

// to check "same group?": findSet(x) == findSet(y)
```

Template C — Build a Weighted Graph (used in Q3, Q4, Q5, Q6)

```
int n, m;
cin >> n >> m;

vector<vector<pair<int,int>>> adj(n);
for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}
```

Template D — Min-Heap Growth (used in Q4, Q5, Q6)

A `priority_queue` in C++ is a max-heap by default. Adding `greater<>` flips it into a min-heap, so the smallest weight always comes out first — exactly what Prim's and Dijkstra need.

```
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
```

```
// pq stores {weight, node}; pq.top() always gives the smallest weight

pq.push({0, startNode});
vector<bool> visited(n, false);

while (!pq.empty()) {
    auto [w, u] = pq.top();
    pq.pop();
    if (visited[u]) continue;    // already handled, skip
    visited[u] = true;

    // ... do something with u here (Prim: add edge, Dijkstra: lock in distance) ...

    for (auto [v, weight] : adj[u]) {
        if (!visited[v]) pq.push({weight, v});
    }
}
```

Notice Templates C and D are almost word-for-word the same in Q4, Q5 and Q6. Once you type this out once, the other two are copy-paste with a tiny change.

Part 3 — The Six Questions

Q1. Activity Selection

Difficulty: ★★ Intermediate — Topic: Greedy, Interval Scheduling

Plain idea

You have a list of activities, each with a start time and an end time. You can only do one activity at a time. You want to fit in as many activities as possible.

The trick: always finish your current activity as early as possible, so you have the most time left over for the next one. That means: sort all activities by their END time, then go through them in that order, taking an activity whenever its start time is not before the end time of the last one you took.

Step-by-step logic

- Read all N activities as (start, end) pairs.
- Sort the activities by end time, smallest first.
- Keep a variable `lastEnd`, starting at -1 (meaning “nothing taken yet”).

- Go through the sorted list. If an activity's start $\geq$ lastEnd, take it: increase the count and set lastEnd to this activity's end.
- If start $<$ lastEnd, skip it — it overlaps with what you already picked.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    // store as {end, start} so sorting the pair sorts by end time automatically
    vector<pair<int,int>> act(n);
    for (int i = 0; i < n; i++) {
        int s, e;
        cin >> s >> e;
        act[i] = {e, s};
    }

    sort(act.begin(), act.end()); // sorted by end time, smallest first

    int count = 0;
    int lastEnd = -1;

    for (int i = 0; i < n; i++) {
        int end = act[i].first;
        int start = act[i].second;
        if (start >= lastEnd) { // does not overlap with the last one taken
            count++;
            lastEnd = end;
        }
    }

    cout << "Max Activities: " << count << endl;
    cout << "Last End: " << lastEnd << endl;
    return 0;
}
```

Walkthrough on Example 1 (input: (1,3) (2,5) (4,7) (6,8))

After sorting by end time the order is already (1,3), (2,5), (4,7), (6,8). lastEnd starts at -1.

- (1,3): start 1 $\geq$ -1 $\rightarrow$ take it. count = 1, lastEnd = 3.

- (2,5): start 2 $\geq$ 3? No $\rightarrow$ skip (overlaps).
- (4,7): start 4 $\geq$ 3 $\rightarrow$ take it. count = 2, lastEnd = 7.
- (6,8): start 6 $\geq$ 7? No $\rightarrow$ skip (overlaps).

Final answer: Max Activities: 2, Last End: 7 — matches the expected output.

Common mistakes a fresher makes here

- Sorting by start time instead of end time — this does NOT always give the best answer.
- Using $>$ instead of $\geq$ when comparing start to lastEnd — back-to-back activities (one starts exactly when the other ends) should be allowed.
- Forgetting to initialise lastEnd to -1, which breaks the very first activity check.

Q2. Minimum Coins

Difficulty: ★★ Intermediate — Topic: Greedy, Change-Making

Plain idea

You want to pay an amount using as few coins as possible. The greedy idea: always use as many of the BIGGEST coin as will fit, then move to the next biggest, and so on. If, after trying every coin, nothing is left over, the amount was payable exactly.

Step-by-step logic

- Read the amount, then read all K coin denominations.
- Sort the denominations from largest to smallest.
- For each denomination (biggest first): while the remaining amount is still $\geq$ that coin, subtract the coin from the remaining amount and add 1 to the coin count.
- After trying every denomination, if remaining amount is 0, the answer is payable exactly; otherwise it is not.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int amount;
```

```

cin >> amount;

int k;
cin >> k;
vector<int> coins(k);
for (int i = 0; i < k; i++) cin >> coins[i];

sort(coins.rbegin(), coins.rend()); // largest denomination first

int remaining = amount;
int coinCount = 0;

for (int i = 0; i < k; i++) {
    while (remaining >= coins[i]) {
        remaining -= coins[i];
        coinCount++;
    }
}

cout << "Coins: " << coinCount << endl;
cout << "Possible: " << (remaining == 0 ? "Yes" : "No") << endl;
return 0;
}

```

Walkthrough on Example 1 (amount = 63, coins = 25 10 5 1)

- Coin 25: 63 → 38 → 13 (used twice, coinCount = 2). 13 < 25, move on.
- Coin 10: 13 → 3 (used once, coinCount = 3). 3 < 10, move on.
- Coin 5: 3 < 5, skip.
- Coin 1: 3 → 2 → 1 → 0 (used three times, coinCount = 6).

Remaining is 0, so Coins: 6, Possible: Yes — matches the expected output.

If your starter code has a bug

The two usual bugs in this kind of starter code are: (1) using if instead of while, so only ONE coin of each value is ever taken instead of as many as fit, and (2) checking `is_possible` by looking at the amount variable instead of what is left over after the greedy loop finishes. Fix both by following the while-loop pattern above and checking `remaining == 0` at the very end.

- Note: greedy coin-picking does not work for every possible set of denominations in general (some odd denomination sets need a different method called Dynamic Programming), but it is exactly the method this lab asks you to implement, and it works for all denominations that include 1.

Q3. Kruskal's MST

Difficulty: ★★★ Advanced — Topic: Minimum Spanning Trees

Plain idea

A Minimum Spanning Tree (MST) connects every node using the cheapest possible total edge weight, with no cycles. Kruskal's algorithm builds it by looking at the cheapest edges first and grabbing any edge that connects two parts of the graph that are not already connected.

“Already connected” is exactly what Union-Find (Template B) answers. So Kruskal's algorithm is really just: sort edges by weight, then loop through them using Union-Find to avoid creating a cycle.

Step-by-step logic

- Read all M edges as (weight, u, v) so they sort by weight automatically.
- Sort the edges from smallest weight to largest.
- Start with every node in its own group (Union-Find).
- Go through the sorted edges. If the two endpoints are in different groups, merge the groups, add the weight to the total, and count the edge.
- If the two endpoints are already in the same group, skip the edge (using it would create a cycle, which a tree never has).
- At the end, the graph is fully connected only if you used exactly $N - 1$ edges.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int parent[1000];

int findSet(int x) {
    if (parent[x] == x) return x;
    return parent[x] = findSet(parent[x]);    // path compression
}

void unionSet(int x, int y) {
    parent[findSet(x)] = findSet(y);
}
```

```
int main() {
    int n, m;
    cin >> n >> m;

    // store as {weight, u, v} so sorting the array sorts by weight first
    vector<array<int,3>> edges(m);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges[i] = {w, u, v};
    }

    sort(edges.begin(), edges.end());

    for (int i = 0; i < n; i++) parent[i] = i;    // each node starts alone

    int totalWeight = 0;
    int edgeCount = 0;

    for (int i = 0; i < m; i++) {
        int w = edges[i][0], u = edges[i][1], v = edges[i][2];
        if (findSet(u) != findSet(v)) {          // different groups: safe to add
            unionSet(u, v);
            totalWeight += w;
            edgeCount++;
        }
    }

    cout << "MST Weight: " << totalWeight << endl;
    cout << "Cables: " << edgeCount << endl;
    cout << "Connected: " << (edgeCount == n - 1 ? "Yes" : "No") << endl;
    return 0;
}
```

Walkthrough on Example 1 (5 nodes, 6 edges)

The six edges sorted from smallest weight to largest are: 0-1 (2), 1-2 (3), 1-4 (5), 0-3 (6), 2-4 (7), 1-3 (8).

- 0-1 (2): different groups → add. Weight = 2, edges = 1.
- 1-2 (3): different groups → add. Weight = 5, edges = 2.
- 1-4 (5): different groups → add. Weight = 10, edges = 3.
- 0-3 (6): different groups → add. Weight = 16, edges = 4.
- 2-4 (7): 2 and 4 already in the same group now → skip.

- 1-3 (8): 1 and 3 already in the same group now → skip.

Final: MST Weight: 16, Cables: 4, Connected: Yes ($4 = N-1 = 5-1$) — matches the expected output.

Common mistakes a fresher makes here

- Forgetting path compression in findSet, which makes the code correct but very slow on big graphs.
- Comparing $u == v$ directly instead of $\text{findSet}(u) == \text{findSet}(v)$ — you must compare group leaders, not the raw node numbers.
- Sizing the parent[] array too small — make sure it is at least as big as N.

Q4. Prim's MST

Difficulty: ★★★ Advanced — Topic: Minimum Spanning Trees

Plain idea

Prim's algorithm also builds a Minimum Spanning Tree, but it grows ONE tree outward from a starting node (vertex 0 here), always extending to the cheapest neighbouring node that is not yet in the tree. This is exactly Template D (min-heap growth).

Step-by-step logic

- Build the adjacency list (Template C). Sort each node's neighbour list so ties are handled the same way every time.
- Push {weight 0, node 0} into a min-heap to start.
- Pop the smallest-weight item. If that node is already visited, skip it and pop again.
- Otherwise mark it visited, add it to the build order, add its weight to the total.
- Push all of its unvisited neighbours into the heap.
- Repeat until the heap is empty.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
```

```

cin >> n >> m;

vector<vector<pair<int,int>>> adj(n);    // adj[u] = {neighbour, weight}
for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}
for (int i = 0; i < n; i++) sort(adj[i].begin(), adj[i].end());

vector<bool> visited(n, false);
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
pq.push({0, 0});    // start at vertex 0, weight 0

int totalWeight = 0;
vector<int> order;

while (!pq.empty()) {
    auto [w, u] = pq.top();
    pq.pop();
    if (visited[u]) continue;
    visited[u] = true;
    order.push_back(u);
    totalWeight += w;

    for (auto [v, weight] : adj[u]) {
        if (!visited[v]) pq.push({weight, v});
    }
}

cout << "MST Weight: " << totalWeight << endl;
cout << "Build Order: ";
for (int i = 0; i < (int)order.size(); i++) {
    cout << order[i];
    if (i != (int)order.size() - 1) cout << " ";
}
cout << endl;
return 0;
}

```

Walkthrough on Example 2 (3 nodes: 0-1 weight 1, 1-2 weight 2, 0-2 weight 3)

- Push {0,0}. Pop {0,0}: visit 0. Order = [0]. Push {1,1} and {3,2}.
- Pop {1,1}: visit 1. Order = [0,1]. Weight = 1. Push {2,2} (the 0-1 link to already-visited 0 is skipped).

- Pop {2,2}: visit 2. Order = [0,1,2]. Weight = 3.

Final: MST Weight: 3, Build Order: 0 1 2 — matches the expected output.

Common mistakes a fresher makes here

- Forgetting the if (visited[u]) continue; check — without it, a node can be added to the tree more than once.
- Using a normal priority_queue without greater<>, which gives a max-heap (largest first) instead of the min-heap Prim's needs.
- Mixing up Prim's (grows ONE tree from a chosen start) with Kruskal's (looks at ALL edges globally, sorted) — they give the same total weight but build it differently.

Q5. Dijkstra Distances

Difficulty: ★★★ Advanced — Topic: Shortest Paths

Plain idea

Dijkstra's algorithm finds the shortest distance from one source node to every other node, when all edge weights are non-negative. It works like Prim's (same min-heap template), except instead of tracking “cheapest edge to grow the tree” it tracks “cheapest known distance from the source.”

The key rule: once a node is popped from the min-heap for the first time, its distance is final and will never get smaller. That is why we mark it done and never update it again.

Step-by-step logic

- Build the adjacency list (Template C).
- Set $\text{dist}[\text{source}] = 0$ and every other $\text{dist}[i] = \text{infinity}$.
- Push {0, source} into a min-heap of {distance, node}.
- Pop the smallest item. If that node is already done, skip it.
- Otherwise mark it done. For each neighbour, if going through this node gives a shorter distance than what the neighbour currently has, update it and push the new {distance, neighbour} into the heap.
- Repeat until the heap is empty. Any node whose distance is still infinity is unreachable.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    int src;
    cin >> src;

    vector<long long> dist(n, LLONG_MAX);
    dist[src] = 0;

    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    pq.push({0, src});

    vector<bool> done(n, false);
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;          // already finalised, skip
        done[u] = true;

        for (auto [v, w] : adj[u]) {
            if (!done[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    int reachable = 0;
    long long farthest = 0;

    cout << "Distances: ";
    for (int i = 0; i < n; i++) {
        if (dist[i] == LLONG_MAX) cout << -1;
        else {
```

```

        cout << dist[i];
        reachable++;
        farthest = max(farthest, dist[i]);
    }
    if (i != n - 1) cout << " ";
}
cout << endl;
cout << "Reachable: " << reachable << endl;
cout << "Farthest: " << farthest << endl;
return 0;
}

```

Walkthrough on Example 2 (4 nodes, source 0)

Edges: 0-1 (1), 1-2 (1), 2-3 (1), 0-3 (10). The direct 0-3 edge costs 10, but going 0→1→2→3 only costs 1+1+1 = 3, so Dijkstra picks the cheaper route.

- $\text{dist} = [0, \infty, \infty, \infty]$. Pop {0,0}, done. Update 1→1, push {1,1}.
- Pop {1,1}, done. Update 2→2, push {2,2}.
- Pop {2,2}, done. Update 3→3 (better than the old ∞), push {3,3}. (Also tries 0-3 = 10, but 3 already beats that and 0 is already done.)
- Pop {3,3}, done.

Final: Distances: 0 1 2 3, Reachable: 4, Farthest: 3 — matches the expected output.

If your starter code has a bug

The most common bug in a Dijkstra skeleton is checking distances on push instead of on pop — i.e. marking a node done as soon as it is added to the heap, instead of when it is popped. This can lock in a distance before a cheaper path is discovered. Always mark done inside the while loop, right after popping, and always check if (done[u]) continue; first.

Q6. Dijkstra Path to Target

Difficulty: ★★★ Advanced — Topic: Shortest Paths

Plain idea

This is the same Dijkstra algorithm as Q5, just asked differently: instead of printing every distance, you only care about ONE target node, plus how many other nodes ended up closer to the source than that target.

Step-by-step logic

- Run the exact same Dijkstra algorithm as Q5 to get `dist[]` for every node.
- If `dist[target]` is still infinity, the target is unreachable: print Distance -1, Reachable No, Closer 0.
- Otherwise, print `dist[target]` and Reachable Yes.
- To count Closer: loop through all nodes except the source, and count how many have a finite distance strictly less than `dist[target]`.

Full working code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    int src, target;
    cin >> src >> target;

    vector<long long> dist(n, LLONG_MAX);
    dist[src] = 0;

    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    pq.push({0, src});

    vector<bool> done(n, false);
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;
        done[u] = true;

        for (auto [v, w] : adj[u]) {
            if (!done[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    if (dist[target] == LLONG_MAX) {
        cout << "-1\n";
    } else {
        cout << dist[target] << "\n";
    }

    int cnt = 0;
    for (int i = 0; i < n; i++) {
        if (i == src) continue;
        if (dist[i] < dist[target]) cnt++;
    }

    cout << cnt << "\n";
}
```

```

    }
  }
}

if (dist[target] == LLONG_MAX) {
  cout << "Distance: -1" << endl;
  cout << "Reachable: No" << endl;
  cout << "Closer: 0" << endl;
} else {
  int closer = 0;
  for (int i = 0; i < n; i++) {
    if (i != src && dist[i] != LLONG_MAX && dist[i] < dist[target]) closer++;
  }
  cout << "Distance: " << dist[target] << endl;
  cout << "Reachable: Yes" << endl;
  cout << "Closer: " << closer << endl;
}
return 0;
}

```

Walkthrough on Example 1 (source 0, target 4)

Running Dijkstra from 0 gives distances 0, 2, 3, 6, 7 for nodes 0, 1, 2, 3, 4.
 $\text{dist}[\text{target}=4] = 7$.

- Node 1: $\text{dist } 2 < 7 \rightarrow \text{closer}$.
- Node 2: $\text{dist } 3 < 7 \rightarrow \text{closer}$.
- Node 3: $\text{dist } 6 < 7 \rightarrow \text{closer}$.

Final: Distance: 7, Reachable: Yes, Closer: 3 — matches the expected output.

Common mistakes a fresher makes here

- Counting the source node itself in Closer — the question says “other than the source,” so always skip $i == \text{src}$.
- Counting unreachable nodes (still infinity) as closer — always check $\text{dist}[i] != \text{LLONG_MAX}$ first.
- Forgetting that if the target is unreachable, Closer must be printed as 0, not skipped.

Part 4 — How to Practice This Without Getting Lost

- Type every template (Part 2) from memory at least three times before touching the questions. The goal is for your hand to know the pattern, not just your eyes.
- For every question, before writing code, say the plain-English idea out loud in one sentence. If you cannot, you are not ready to code it yet — re-read the “Plain idea” section.
- Always trace your code by hand on the small Example 2 in each question first. It is smaller and easier to track on paper than Example 1.
- If your answer does not match, print `dist[]`, `parent[]`, or `order[]` in the middle of the loop to see where it goes wrong, instead of staring at the whole code at once.
- Q1 and Q2 (Greedy) are the easiest — master those first. Q3 and Q4 (MST) come next since they reuse the graph-building template. Q5 and Q6 (Dijkstra) are really the same algorithm asked two ways, so once one works, the other is mostly copy-paste.

Good luck — once Template C and D feel automatic, most graph problems in this style stop feeling new and start feeling familiar.